

Primer avance proyecto física computacional.

Benjamín Medina¹, *benjamin.medinas@usm.cl*

¹*Departamento de Física, Universidad Técnica Federico Santa María, San Joaquín, Chile*

1. Introducción y Alcance del Modelo

El presente proyecto tiene como objetivo la simulación computacional de la propagación de ondas sísmicas bidimensionales en medios geológicos heterogéneos. El desarrollo completo del proyecto contempla 8 hitos principales, desde la construcción del solver diferencial hasta la inversión tomográfica.

Este primer informe de avance documenta la etapa preliminar: la formulación físico-matemática del problema, el esquema numérico utilizado y el desarrollo inicial del solver espacial para simular el frente de onda originado por una fuente sísmica (Pulso de Ricker) a través de un modelo de velocidad variable.

2. Física del Problema: Modelo Matemático

El fenómeno de propagación sísmica en un medio bidimensional continuo y heterogéneo (donde la velocidad de propagación varía en el espacio) se modela de manera simplificada mediante la ecuación de onda acústica escalar:

$$\frac{\partial^2 u}{\partial t^2} = c^2(x, y) \nabla^2 u = c^2(x, y) \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (1)$$

donde $u(x, y, t)$ representa el campo de desplazamiento escalar (o presión acústica) y $c(x, y)$ es el campo de velocidades del medio, introducido al modelo a partir de un archivo de capas geológicas.

El marco teórico de referencia está dado por el trabajo de Virieux (1986). Si bien Virieux formula un sistema elastodinámico acoplado de velocidad-esfuerzo, la lección fundamental de su investigación es el tratamiento de discontinuidades abruptas (interfaces entre distintas capas geológicas). Tal como demuestra Virieux, la estabilidad en medios altamente heterogéneos requiere que las variables espaciales de los parámetros del medio estén correctamente discretizadas en la malla para evitar dispersión e inestabilidad numérica en las zonas de contraste de velocidad.

2.1. Fuente Sísmica: Pulso de Ricker

Para modelar la fuente sísmica que excita el medio, se utiliza un pulso de Ricker, que corresponde a la segunda derivada de una función Gaussiana. Su forma analítica en el dominio del tiempo es:

$$f(t) = (1 - 2\pi^2 f_p^2 (t - t_0)^2) e^{-\pi^2 f_p^2 (t - t_0)^2} \quad (2)$$

donde f_p es la frecuencia central dominante de la onda (típicamente entre 10 y 30 Hz en sismología) y t_0 es el tiempo de retardo temporal que asegura que la señal sea causal (nula para $t < 0$).

3. Métodos Numéricos: Diferencias Finitas de 4to Orden

Para resolver numéricamente la ecuación (1), se discretiza el dominio computacional en una malla espacial de paso $\Delta x = \Delta y$ y un paso temporal Δt . La aproximación de la segunda derivada espacial requerida en el proyecto se realiza mediante un esquema en Diferencias Finitas centradas de cuarto orden en el espacio ($O(\Delta x^4)$), garantizando menor dispersión numérica al propagar ondas de alta frecuencia:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{-u_{i+2,j}^n + 16u_{i+1,j}^n - 30u_{i,j}^n + 16u_{i-1,j}^n - u_{i-2,j}^n}{12\Delta x^2} \quad (3)$$

Análogamente se desarrolla para la derivada en el eje y (subíndice j). Para la integración temporal, se emplea un esquema explícito de segundo orden ($O(\Delta t^2)$):

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} \quad (4)$$

Sustituyendo y despejando el campo en el futuro ($u_{i,j}^{n+1}$), se obtiene la regla de actualización explícita que gobierna la simulación. La estabilidad computacional de este esquema explícito está condicionada por el criterio CFL (Courant-Friedrichs-Lewy), que exige que el paso temporal sea suficientemente pequeño respecto a la velocidad máxima del medio geológico: $\Delta t \leq \frac{h}{c_{max}\sqrt{2}}$.

4. Implementación Preliminar (Avance)

Se presenta el núcleo preliminar del solver programado en Python, que establece las matrices y el loop principal de la ecuación de onda.

```
1 import numpy as np
2
3 def ricker_pulse(f_central, dt, n_steps, t0):
4     t = np.arange(n_steps) * dt
5     arg = np.pi * f_central * (t - t0)
6     return (1.0 - 2.0 * arg**2) * np.exp(-arg**2)
7
8 nx, ny = 200, 200
9 dx, dy = 10.0, 10.0
10 dt = 0.001
11 n_steps = 1000
12
13 u_prev = np.zeros((nx, ny))
14 u_curr = np.zeros((nx, ny))
15 u_next = np.zeros((nx, ny))
16
17 C = np.ones((nx, ny)) * 2000.0
18
19 alpha = (C * dt / dx)**2
20
21 for n in range(1, n_steps):
22
23     laplaciano = (
24         - u_curr[4:, 2:-2] + 16 * u_curr[3:-1, 2:-2] - 30 * u_curr[2:-2, 2:-2] + 16 *
25         u_curr[1:-3, 2:-2] - u_curr[:-4, 2:-2] +
```

```
25     - u_curr[2:-2, 4:] + 16 * u_curr[2:-2, 3:-1] - 30 * u_curr[2:-2, 2:-2] + 16 *  
26     u_curr[2:-2, 1:-3] - u_curr[2:-2, :-4]  
27 ) / 12.0  
28 u_next[2:-2, 2:-2] = 2 * u_curr[2:-2, 2:-2] - u_prev[2:-2, 2:-2] + alpha[2:-2, 2:-2]  
29     * laplaciano  
30 u_prev[:, :] = u_curr[:, :]  
31 u_curr[:, :] = u_next[:, :]
```